

FIT3155 S2/2026: Assignment 1

(Due date: 11:55pm Friday 18 Sept 2026)

[Weight: $20 = 7 + 6 + 7$ marks.]

Assessment guidelines:

- You **must use the Git repository provided for the assignment** for all implementation work and push your work regularly throughout the assignment period. (See separate section explain this below.)
- You **must write all code yourself**. Use of Generative AI tools is **not** allowed.
- Any code **derived from online sources must be declared and fully cited** to avoid academic integrity investigation.
- Assessment is about your **understanding and ability to write code yourself**.
- You may be **called for an interview** to justify your understanding of your code.
- You should not use **external library routines**, except those considered standard.
 - Standard data structures (such as lists, dictionaries, tuples, and sets) are allowed if they do not conflict with the assessment objectives. Ensure their use is appropriate and efficient in terms of space and time.

Submission instructions:

- **Git push:** Your Git repository should show your complete code-writing history through regular pushes (see the section below).
- **Moodle submission:** In addition to regularly pushing your work to the provided Git repository, you must submit a **.zip** archive of your completed repository via Moodle. The archive must contain the original directory `<student_ID>/`, with the subdirectories and corresponding files `q1/a1q1.py`, `q2/a1q2.py`, and `q3/a1q3.py`. The zip file should be named `<student_ID>.zip`. This should be uploaded before the deadline. Late submissions will incur penalties as per Monash policy.

Academic integrity

Monash University's academic integrity policy is accessible via this link: <https://www.monash.edu/students/academic/policies/academic-integrity> (click). Read it carefully to understand your responsibilities. **As per FIT policy, all submissions will be scanned via MOSS (click) and/or JPlag (click).**

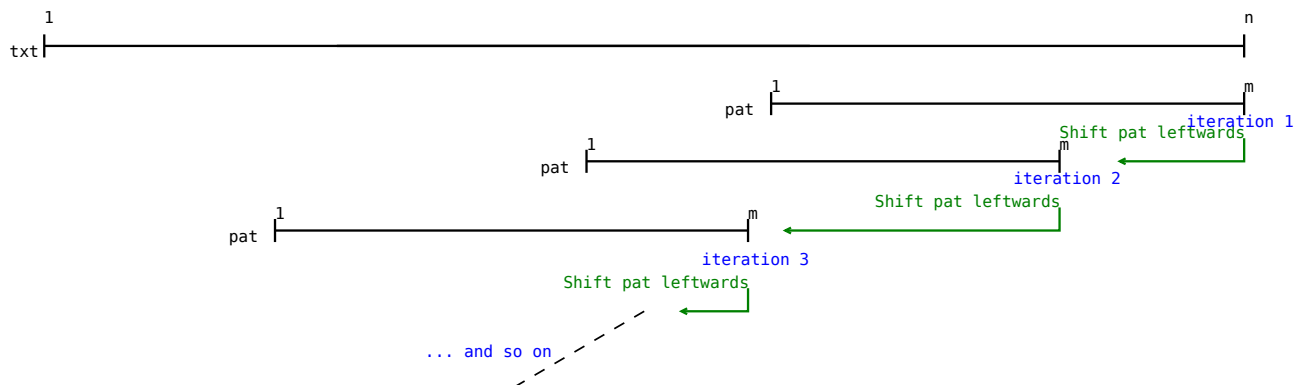
Git repository evidence requirement

Follow the ‘FIT3155 Instructions for Using GitLab’ document available on Moodle under Assessments tab (click on Assignment 1 link). You must complete cloning your git repos as soon as the assignment spec is released. It is an expectation that you record the continual coding progress of your individual attempt throughout the assignment period. Your Git history will form an integral part of the review and marking of your submission. If your commit history is inconclusive, you will be called in for a further interview during which you will be asked to code live to demonstrate your understanding of what you submitted. Forgetting to push to FIT Git repository is not an acceptable excuse.

It is expected that, while working on the assignment, you make **at least two commits and one push per hour of active work** to ensure that your repository maintains a coherent record of your code development. *Active work* refers to periods during which you are actually working on the assignment. Your commits and pushes should represent a clear progression of development. Natural commit points include implementing a logical block of code, modifying or extending existing functionality, fixing bugs, completing testing, amongst others. Artificially creating trivial changes solely to increase the number of commits or pushes does not satisfy this requirement.

Assignment Questions

1. **Variant implementation of the Boyer-Moore algorithm:** In this question, you will have to implement a version of Boyer-Moore that shifts the pattern **leftwards** along the text between iterations, starting with the initial iteration where the ends of the text and pattern are aligned (see illustration below):



(Note: You are NOT allowed to simply reverse the text and pattern strings and run the Boyer-Moore algorithm in the regular direction.)

In this variant of Boyer-Moore, you will need to construct analogues of the extended bad-character and good suffix/matched prefix shift rules covered in Week 2 for the regular implementation. Any preprocessed data structures must be fully analogous to those covered in the Week 2 lecture. Your code implementing these components should be modular. That is, there should be distinct functions to compute the analogues of the extended bad-character table, good suffix array, and matched prefix array for this problem. Each of these data structures should strictly store values analogous to those computed in the examples shown in the lecture slides, except that your implementation should store all values for analogous data structures using 0-based indexing.

All efficiency considerations that are applicable to the regular Boyer-Moore algorithm should also be applied to this variant.

Strictly follow the following specification to address this question:

Program name: `a1q1.py`

Arguments to your program: Two filenames:

- (a) First containing `txt[1...n]` string (without any line breaks).
- (b) Second containing `pat[1..m]` string (without any line breaks).

For this question, all characters in the text and pattern will be within the ASCII range [97, 122]. You are free to assume that the pattern and text are non-empty and that the pattern is equal to or shorter than the text.

Command line usage of your script:

```
python a1q1.py <text filename> <pattern filename>
```

Do not hard-code the input filenames within your program. Ensure your program can be executed from the terminal (command line) by supplying any pair of filenames (containing text and pattern strings) as command-line arguments. Include sufficient inline comments explaining each logical block of your code. Code that lacks adequate comments will require a face-to-face interview to verify your understanding before it can be assessed.

Output file name: `output_a1q1.txt`

- Each position where `pat` matches `txt` should appear on a separate line in the output file. This implementation will naturally report the positions in reverse order. For example, when `txt[1...25] = aabbabababbbbbaabaabbabbaa`, and `pat[1...3] = aba`, the output should be

```
15
7
5
```

No further test cases will be provided. You will have to develop test cases yourself.

2. **Constrained regular-expression (regex) matching using Burrows-Wheeler Transform (BWT):** In this task, you are asked to extend BWT-based exact pattern matching to support regex pattern matching for the following limited set of regex constructs:

Fixed/Literal character: A fixed/literal character in the pattern must match the same character in the text. For example, the pattern `abc` matches only the substring `abc` at any position in the text where it occurs.

Dot (.) symbol: The dot symbol in the pattern matches any single character in the text. For example, the pattern `x.d` can match `xad`, `xbd`, `xcd`, and so on, wherever they occur in the text.

Negated character-group [`^characters`]: A character in the specified group is **not** allowed to match the corresponding character in the text. For example, the pattern `x[^kc]d` does not allow the second character of any matched substring of the text to be either `k` or `c`.

Your task for this question is to implement a parser for the constrained regex syntax described above and to *efficiently* perform regex pattern matching using the BWT of the given text as the search index. The parser should process the input pattern and represent each position of the pattern according to its corresponding construct (literal character, dot symbol, or negated character group). The resulting representation should then be used by your pattern-matching algorithm to search for all occurrences of the pattern in the text using the BWT.

For this question, you may construct the BWT naively by explicitly sorting text suffixes (and you are allowed to use Python's built-in `sort`).*

In your implementation, you may assume that the pattern and text are non-empty ($\text{length} > 0$), that the pattern is equal to or shorter than the text, and that every input pattern conforms to the constrained regex syntax specified above (you do **not** have to consider any other regex constructs).

Your submitted code should be fully modular, with separate functions for each logical task. Spaghetti code that is written in an unclear or unnecessarily complicated manner will incur penalties.

Strictly follow the following specification to address this question:

Program name: `a1q2.py`

Arguments to your program: Two filenames:

- (a) First containing the text string (fully composed of fixed/literal characters, without any line breaks).
- (b) Second containing the pattern regex (composed of the regex constructs defined above, also without any line breaks).

All fixed/literal characters in the text and pattern will be within the ASCII range [97, 122]

Command line usage of your script:

```
python a1q2.py <text filename> <pattern filename>
```

Do not hard-code the input filenames within your program. Ensure your program can be executed from the terminal by supplying any pair of filenames (containing text and pattern strings) as command-line arguments. Include sufficient inline comments explaining each logical block of your code. Code that lacks adequate comments will require a face-to-face interview to verify your understanding before it can be assessed.

Output file name: `output_a1q2.txt`

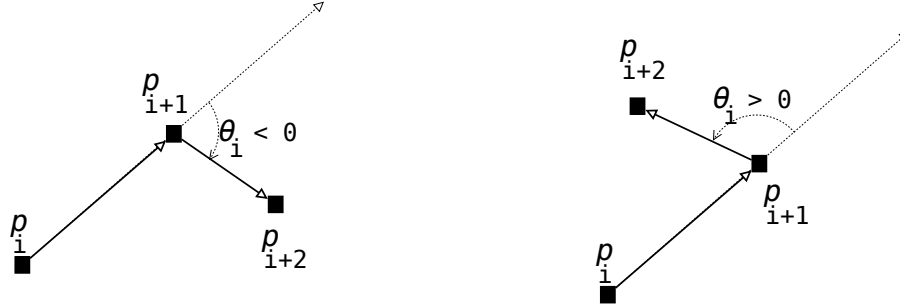
- Each position where `pat` matches a substring of `txt` should appear on a separate line in the output file.
- Positions should be reported using 1-based indexing and should be listed in increasing order.
- A match occurs at a position in the text, when the corresponding substring starting at that position matches the constrained regex pattern.
- For example, if the text is `txt[1...22] = abcdcbadbddabbddfabcd` and the pattern regex is `.b[^ac]d` then the output file will contain:

```
8
12
13
```

3. [Matching discrete angular states](#) Read and analyse the problem description given below, and determine which of the three algorithms covered as main topics in weeks 1-3 is most suited for solving that problem, while taking into consideration their respective worst-case time and space requirements. After making this choice, you must implement your chosen algorithm to solve the stated problem.

*Note: Although not required for this assignment, just for your information, Ukkonen's suffix tree construction allows you to compute the BWT of a string in time linearly proportional to its length. The efficiency of the BWT construction will not be assessed for this assignment. Efficiency will only be a point of consideration for the regex pattern matching part using the constructed BWT as a search index.

You will be given a sequence of two-dimensional (x, y) points $\{p_1, p_2, \dots, p_n\}$. From these given points, your program should compute a sequence of signed angles $\{\theta_1, \theta_2, \dots, \theta_{n-2}\}$, with one signed angle for each set of three consecutive points p_i, p_{i+1}, p_{i+2} , for all $1 \leq i \leq n-2$. Specifically, each θ_i is the signed angle formed by the vectors $\overrightarrow{p_i p_{i+1}}$ and $\overrightarrow{p_{i+1} p_{i+2}}$, measured in the range $(-180^\circ, 180^\circ]$. Positive angles represent anticlockwise rotations, while negative angles represent clockwise rotations.



An (integer) angular discretization parameter $k \geq 1$ will also be given as input to your program. This parameter divides the angular range $(-180^\circ, 180^\circ]$ into k equal sized angular bins. Each observed signed angle θ_i is assigned to exactly one of these bins, producing an unambiguous discrete angular-state sequence $s_1, s_2, \dots, s_{n-2}$, where each state is an integer in $\{1, \dots, k\}$. For example, when $k = 2$, the two bins are $(-180^\circ, 0^\circ]$ corresponding to state represented by 1, and $(0^\circ, 180^\circ]$ corresponding to a state represented by 2.

You will also be given a query as a sequence of integer states $q_1, q_2, \dots, q_m$, where each query state q_j belongs to $\{0, 1, \dots, k\}$. A positive query state must exactly match the corresponding angular state, while a query state of 0 is allowed to match any angular state.

Your goal is to find all positions where the query matches a consecutive subsequence of the angular-state sequence (of the same sequence length). In other words, for a query sequence $q_1, q_2, \dots, q_m$, report all indices i satisfying $1 \leq i \leq n - m - 1$ such that, for every $1 \leq j \leq m$, $q_j = 0$ or $q_j = s_{i+j-1}$.

As before, write your code in a modular manner, with separate functions for each logical task.

Strictly follow the following specification to address this question:

Program name: a1q3.py

Arguments to your program: Three command-line arguments:

- (a) the filename containing the sequence of 2D points (one point per line, space separated),
- (b) the integer $k \geq 1$, specifying the number of angular states, and
- (c) the filename containing the query sequence of states (one per line)

(You are free to assume that the points file and query sequence of states are non-empty and that the query sequence of states (one per line) is no larger in length than the sequence of discretized states generated from the input sequence of points.)

Command-line usage of your script:

`python a1q3.py <points filename> <k> <query filename>` Do not hard-code the input filenames or the value of k within your program. The program must accept any valid pair of filenames and any valid value of k supplied through the command line. Include sufficient inline comments explaining each logical block of your code. Code that lacks adequate comments will require a face-to-face interview to verify your understanding before it can be assessed.

Output file name: output_a1q3.txt

- Write each matching position on a separate line.
- A match beginning at angular state s_i should be reported as (1-based) position i in the original point sequence.
- Report all matching positions in increasing order.
- If there are no matches, the output file should be created and should be empty.

Example:

Points:	k = 4	Query:	Output:
2.562 14.92			9
4.982 15.288		1	23
3.564 13.986		4	
4.475 15.017		0	
4.651 12.596		3	
4.511 14.011		3	
5.889 12.375		2	
4.687 11.711		0	
0.938 11.579			
0.208 12.517			
2.398 13.309			
1.539 13.829			
3.576 16.478			
3.698 18.633			
3.798 22.4			
7.472 23.497			
8.7 20.532			
10.582 20.991			
9.29 19.04			
10.281 18.477			
7.926 17.367			
9.098 16.199			
7.885 16.745			
9.36 14.541			
8.8 15.002			
8.766 12.209			
10.706 14.478			
12.988 17.519			
10.847 20.606			
13.72 23.113			
12.807 24.722			
9.31 25.905			

--o0o--
END
--o0o--